InternNova

Java Development

Name: Holehonnur Ramachandra Anagha

Week 6 Final Project: Java Application Development

Introduction: This project is a Student Management System that runs in the console. It has a menu where we can add, view, search, update and delete student records. I also made it save the data in a text file, so the records are still there when the program is closed and opened again.

Objective: The objective of this final project is to apply the Java concepts learned throughout the 6-week internship and develop a functional Java application. Students will also learn to manage and submit their project using Git and GitHub.

Features:

- Add, view, update and delete student records
- Search a student by ID or by name
- Student ID can't repeat, and the grade is worked out from the marks
- Shows the list of unique courses
- Saves the data in students.txt and loads it again when the program starts
- Makes a backup file on exit, using a separate thread
- Shows an error message for wrong input instead of crashing

Concepts Used

Classes and objects, constructors, this and static, inheritance (Student extends Person), polymorphism (overriding and overloading), encapsulation, abstraction (abstract class Person and interface RecordOperations), exception handling, ArrayList, HashSet, HashMap, StringBuilder, file handling and multithreading.

Project Structure:

Person: the abstract parent class with name and age

Student: the student details, which extends Person

RecordOperations: the interface with the list of operations

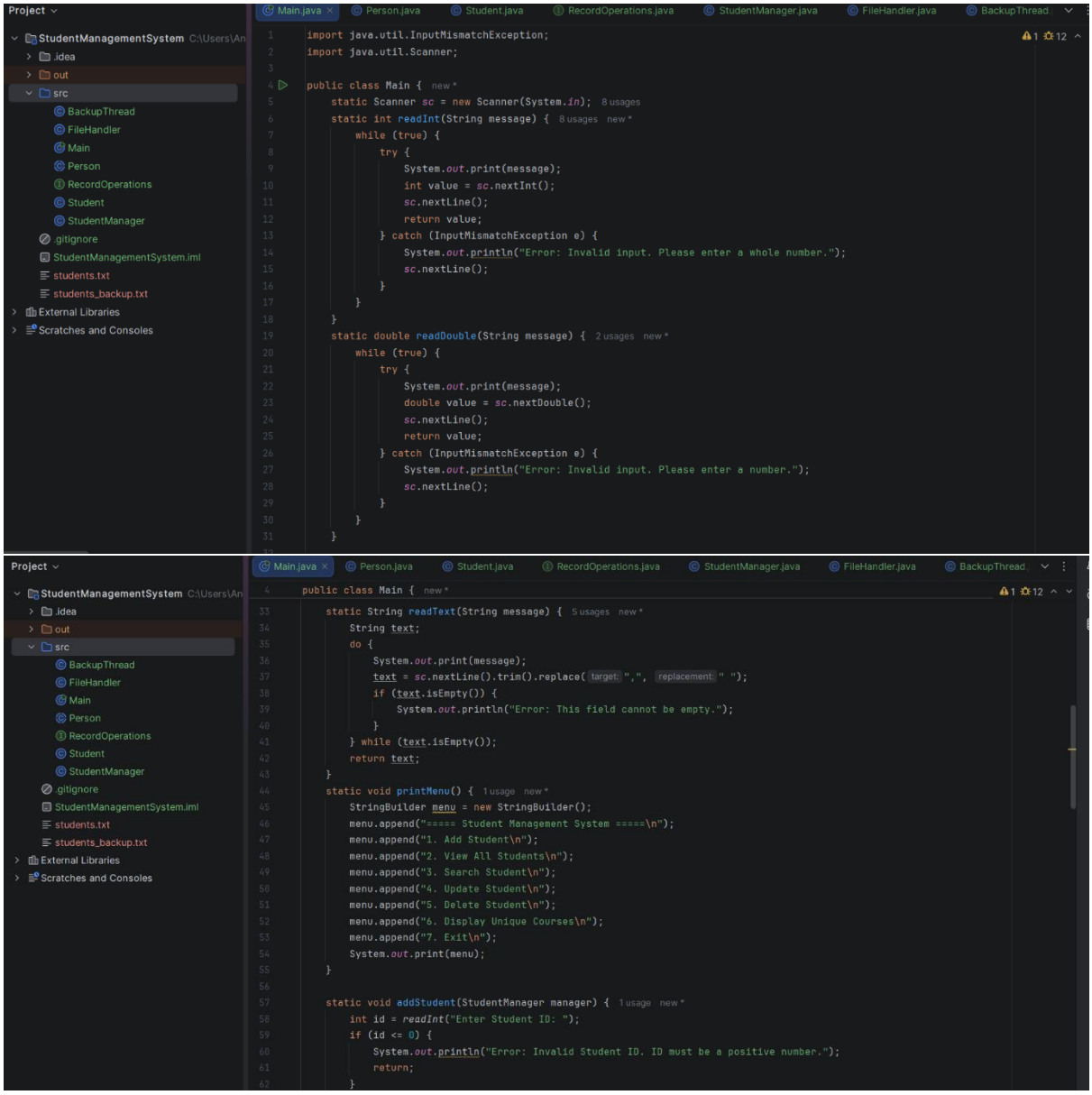
StudentManager: stores the students in a HashMap and does the add, search, update and delete work

FileHandler: saves the students to the file and reads them back

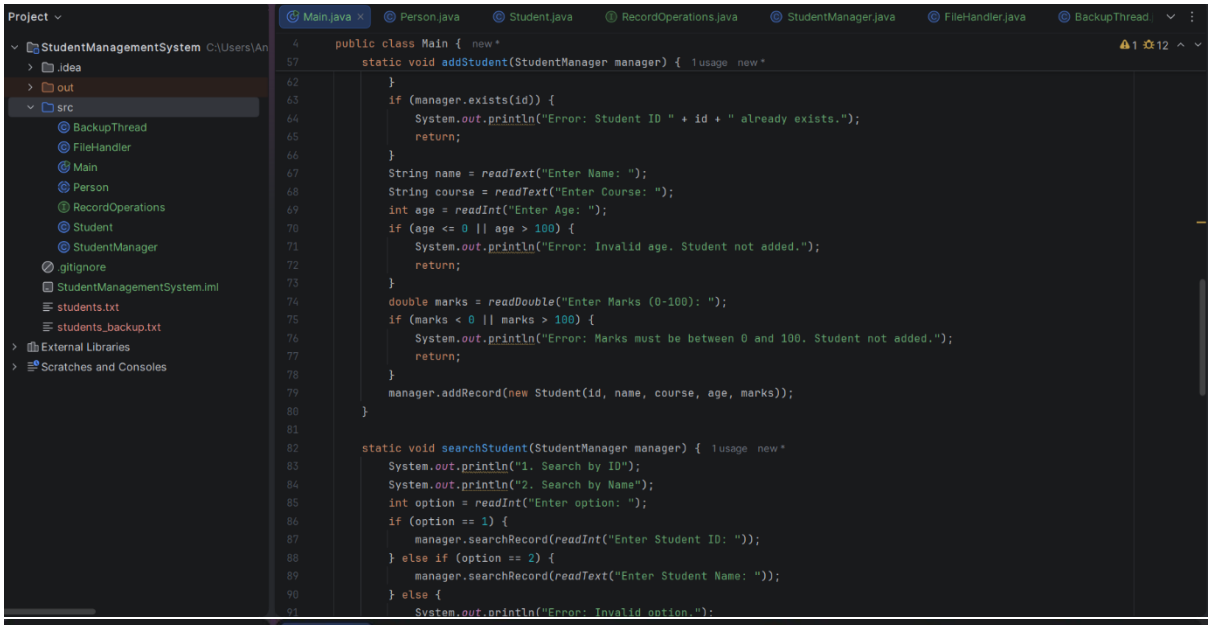
BackupThread: makes the backup file in a separate thread

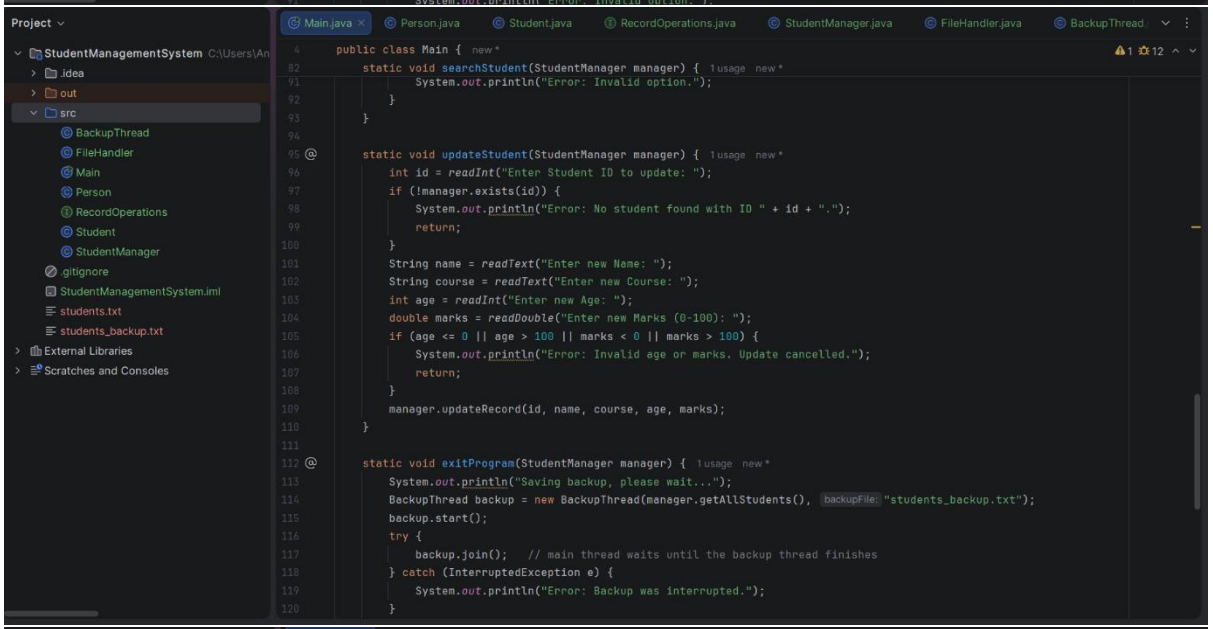
Main: shows the menu and takes the user input

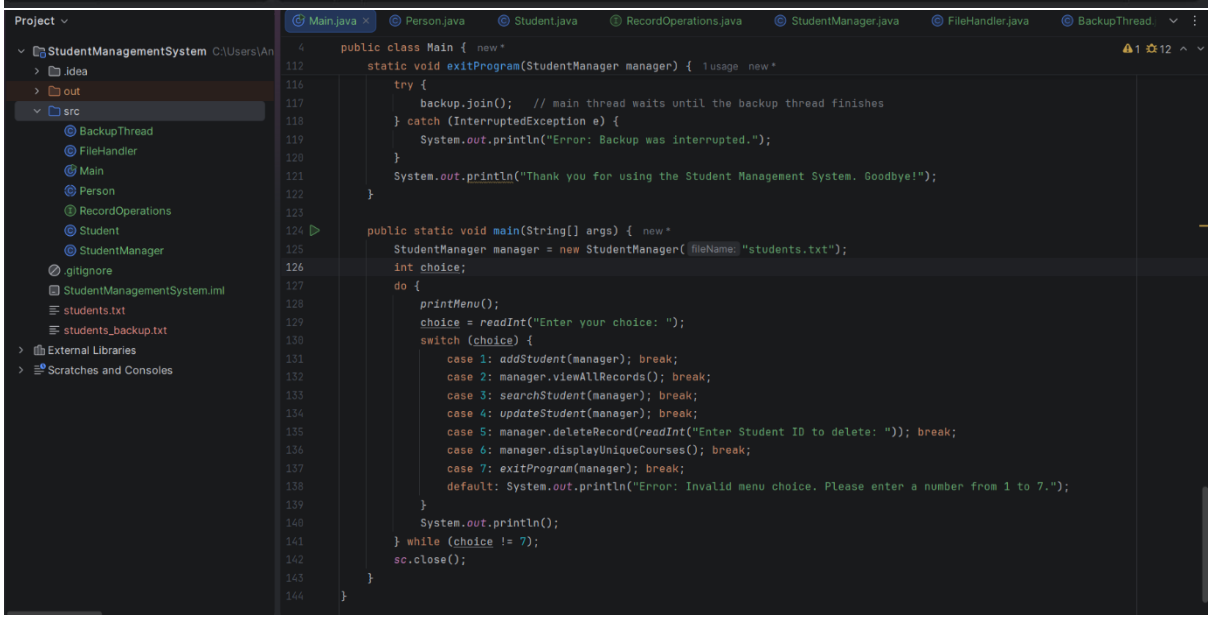
Main.java


Project
StudentManagementSystem C:\Users\An
.idea
out
src
BackupThread
FileHandler
Main
Person
RecordOperations
Student
StudentManager
.gitignore
StudentManagementSystem.iml
students.txt
students_backup.txt
External Libraries
Scratches and Consoles
Main.java
Person.java
Student.java
RecordOperations.java
StudentManager.java
FileHandler.java
BackupThread
import java.util.InputMismatchException;
import java.util.Scanner;
public class Main { new *
static Scanner sc = new Scanner(System.in); 8 usages
static int readInt(String message) { 8 usages new *
while (true) {
try {
System.out.print(message);
int value = sc.nextInt();
sc.nextLine();
return value;
} catch (InputMismatchException e) {
System.out.println("Error: Invalid input. Please enter a whole number.");
sc.nextLine();
}
}
}
static double readDouble(String message) { 2 usages new *
while (true) {
try {
System.out.print(message);
double value = sc.nextDouble();
sc.nextLine();
return value;
} catch (InputMismatchException e) {
System.out.println("Error: Invalid input. Please enter a number.");
sc.nextLine();
}
}
}

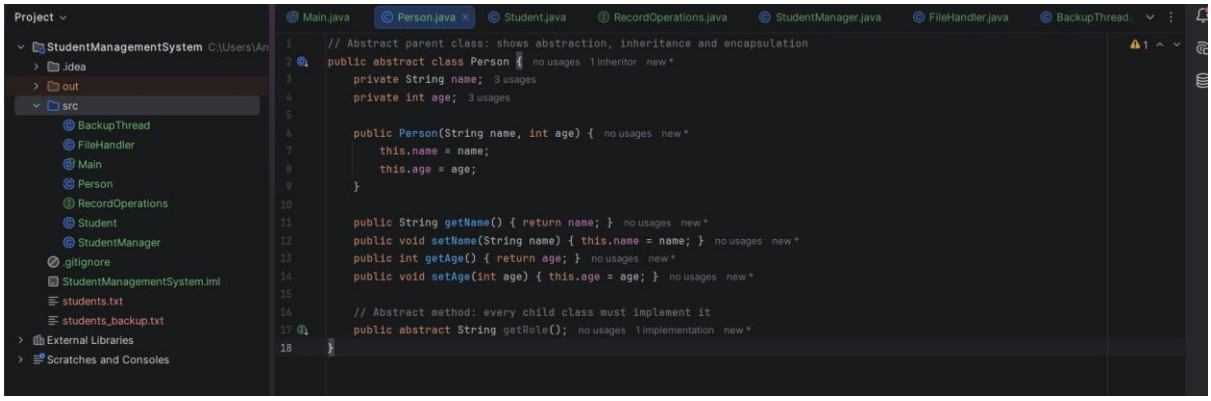
Project
StudentManagementSystem C:\Users\An
.idea
out
src
BackupThread
FileHandler
Main
Person
RecordOperations
Student
StudentManager
.gitignore
StudentManagementSystem.iml
students.txt
students_backup.txt
External Libraries
Scratches and Consoles
Main.java
Person.java
Student.java
RecordOperations.java
StudentManager.java
FileHandler.java
BackupThread
public class Main { new *
static String readText(String message) { 5 usages new *
String text;
do {
System.out.print(message);
text = sc.nextLine().trim().replace(target: ",", replacement: " ");
if (text.isEmpty()) {
System.out.println("Error: This field cannot be empty.");
}
} while (text.isEmpty());
return text;
}
static void printMenu() { 1 usage new *
StringBuilder menu = new StringBuilder();
menu.append("===== Student Management System =====\n");
menu.append("1. Add Student\n");
menu.append("2. View All Students\n");
menu.append("3. Search Student\n");
menu.append("4. Update Student\n");
menu.append("5. Delete Student\n");
menu.append("6. Display Unique Courses\n");
menu.append("7. Exit\n");
System.out.print(menu);
}
static void addStudent(StudentManager manager) { 1 usage new *
int id = readInt("Enter Student ID: ");
if (id <= 0) {
System.out.println("Error: Invalid Student ID. ID must be a positive number.");
return;
}


Project
StudentManagementSystem C:\Users\An
.idea
out
src
BackupThread
FileHandler
Main
Person
RecordOperations
Student
StudentManager
.gitignore
StudentManagementSystem.iml
students.txt
students_backup.txt
External Libraries
Scratches and Consoles
Main.java
Person.java
Student.java
RecordOperations.java
StudentManager.java
FileHandler.java
BackupThread.
public class Main { new *
static void addStudent(StudentManager manager) { 1 usage new *
}
if (manager.exists(id)) {
System.out.println("Error: Student ID " + id + " already exists.");
return;
}
String name = readText("Enter Name: ");
String course = readText("Enter Course: ");
int age = readInt("Enter Age: ");
if (age <= 0 || age > 100) {
System.out.println("Error: Invalid age. Student not added.");
return;
}
double marks = readDouble("Enter Marks (0-100): ");
if (marks < 0 || marks > 100) {
System.out.println("Error: Marks must be between 0 and 100. Student not added.");
return;
}
manager.addRecord(new Student(id, name, course, age, marks));
}
static void searchStudent(StudentManager manager) { 1 usage new *
System.out.println("1. Search by ID");
System.out.println("2. Search by Name");
int option = readInt("Enter option: ");
if (option == 1) {
manager.searchRecord(readInt("Enter Student ID: "));
} else if (option == 2) {
manager.searchRecord(readText("Enter Student Name: "));
} else {
System.out.println("Error: Invalid option.");

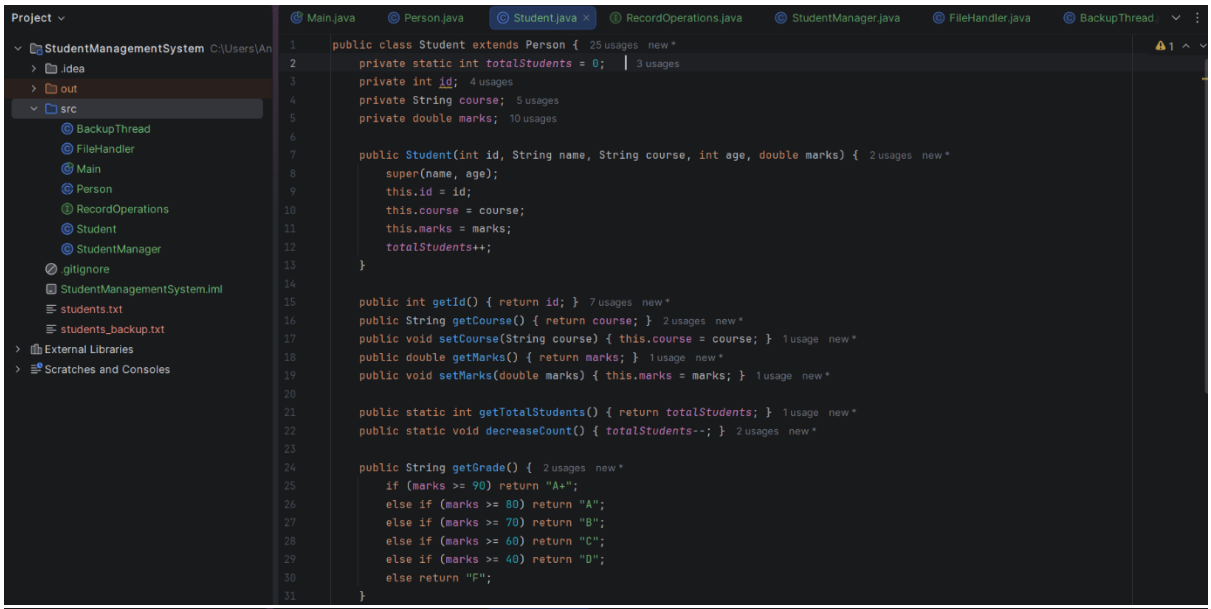

Project
StudentManagementSystem C:\Users\An
.idea
out
src
BackupThread
FileHandler
Main
Person
RecordOperations
Student
StudentManager
.gitignore
StudentManagementSystem.iml
students.txt
students_backup.txt
External Libraries
Scratches and Consoles
Main.java
Person.java
Student.java
RecordOperations.java
StudentManager.java
FileHandler.java
BackupThread.
public class Main { new *
System.out.println("Error: Invalid option.");
}
}
static void updateStudent(StudentManager manager) { 1 usage new *
int id = readInt("Enter Student ID to update: ");
if (!manager.exists(id)) {
System.out.println("Error: No student found with ID " + id + ".");
return;
}
String name = readText("Enter new Name: ");
String course = readText("Enter new Course: ");
int age = readInt("Enter new Age: ");
double marks = readDouble("Enter new Marks (0-100): ");
if (age <= 0 || age > 100 || marks < 0 || marks > 100) {
System.out.println("Error: Invalid age or marks. Update cancelled.");
return;
}
manager.updateRecord(id, name, course, age, marks);
}
static void exitProgram(StudentManager manager) { 1 usage new *
System.out.println("Saving backup, please wait...");
BackupThread backup = new BackupThread(manager.getAllStudents(), backupFile: "students_backup.txt");
backup.start();
try {
backup.join(); // main thread waits until the backup thread finishes
} catch (InterruptedException e) {
System.out.println("Error: Backup was interrupted.");
}


Project
StudentManagementSystem C:\Users\An
.idea
out
src
BackupThread
FileHandler
Main
Person
RecordOperations
Student
StudentManager
.gitignore
StudentManagementSystem.iml
students.txt
students_backup.txt
External Libraries
Scratches and Consoles
Main.java
Person.java
Student.java
RecordOperations.java
StudentManager.java
FileHandler.java
BackupThread.
public class Main { new *
static void exitProgram(StudentManager manager) { 1 usage new *
try {
backup.join(); // main thread waits until the backup thread finishes
} catch (InterruptedException e) {
System.out.println("Error: Backup was interrupted.");
}
System.out.println("Thank you for using the Student Management System. Goodbye!");
}
public static void main(String[] args) { new *
StudentManager manager = new StudentManager(fileName: "students.txt");
int choice;
do {
printMenu();
choice = readInt("Enter your choice: ");
switch (choice) {
case 1: addStudent(manager); break;
case 2: manager.viewAllRecords(); break;
case 3: searchStudent(manager); break;
case 4: updateStudent(manager); break;
case 5: manager.deleteRecord(readInt("Enter Student ID to delete: ")); break;
case 6: manager.displayUniqueCourses(); break;
case 7: exitProgram(manager); break;
default: System.out.println("Error: Invalid menu choice. Please enter a number from 1 to 7.");
}
System.out.println();
} while (choice != 7);
sc.close();
}
}

Person.java

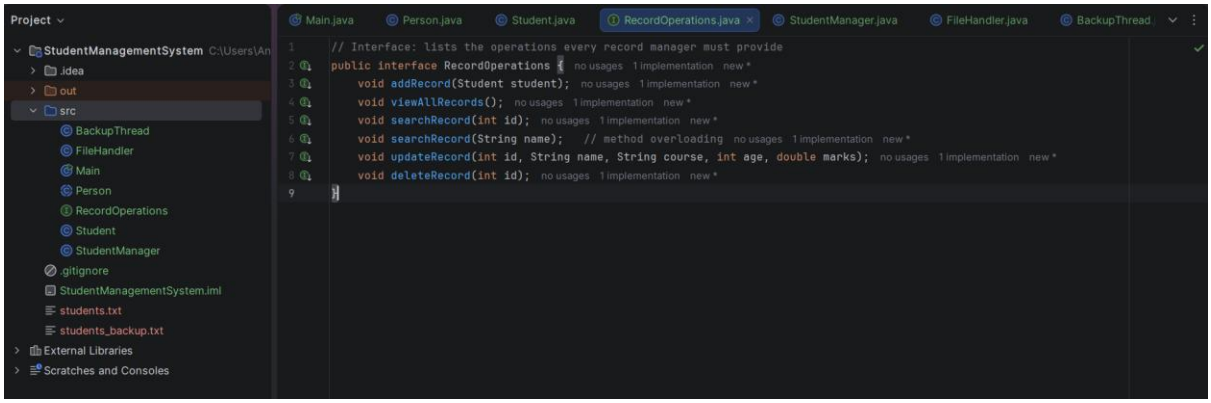


Student.java



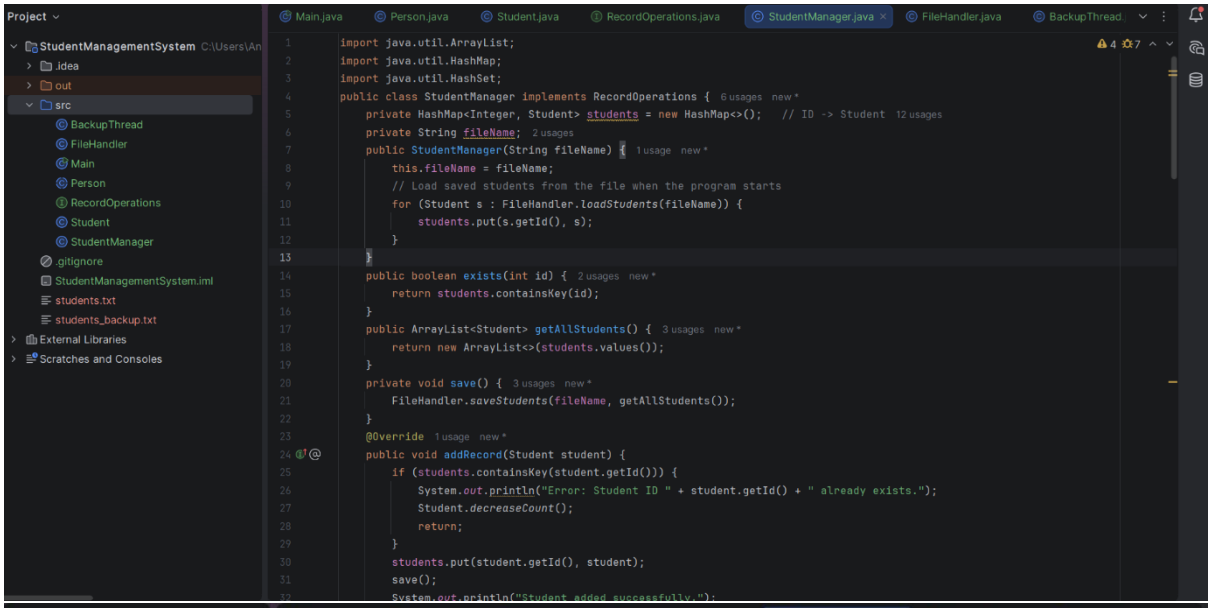


RecordOperation.java

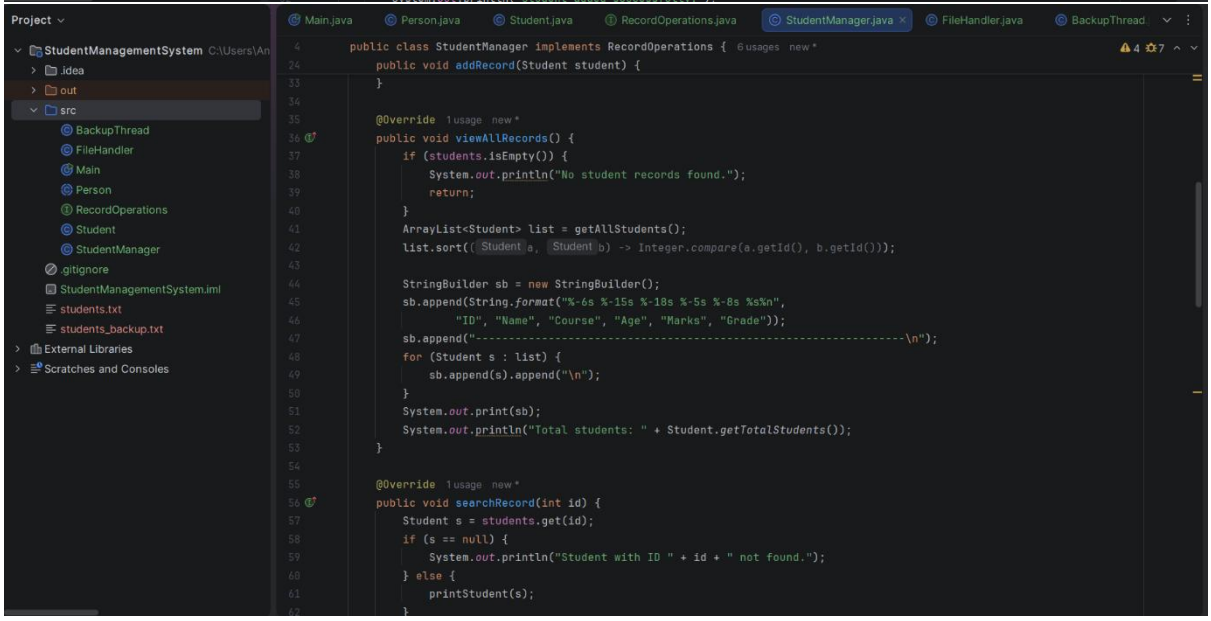


```
// Interface: lists the operations every record manager must provide
public interface RecordOperations {
    void addRecord(Student student);
    void viewAllRecords();
    void searchRecord(int id);
    void searchRecord(String name);   // method overloading
    void updateRecord(int id, String name, String course, int age, double marks);
    void deleteRecord(int id);
}
```

StudentManager.java



```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
public class StudentManager implements RecordOperations {
    private HashMap<Integer, Student> students = new HashMap<>();   // ID -> Student
    private String fileName;
    public StudentManager(String fileName) {
        this.fileName = fileName;
        // Load saved students from the file when the program starts
        for (Student s : FileHandler.loadStudents(fileName)) {
            students.put(s.getId(), s);
        }
    }
    public boolean exists(int id) {
        return students.containsKey(id);
    }
    public ArrayList<Student> getAllStudents() {
        return new ArrayList<>(students.values());
    }
    private void save() {
        FileHandler.saveStudents(fileName, getAllStudents());
    }
    @Override
    public void addRecord(Student student) {
        if (students.containsKey(student.getId())) {
            System.out.println("Error: Student ID " + student.getId() + " already exists.");
            Student.decreaseCount();
            return;
        }
        students.put(student.getId(), student);
        save();
        System.out.println("Student added successfully.");
```

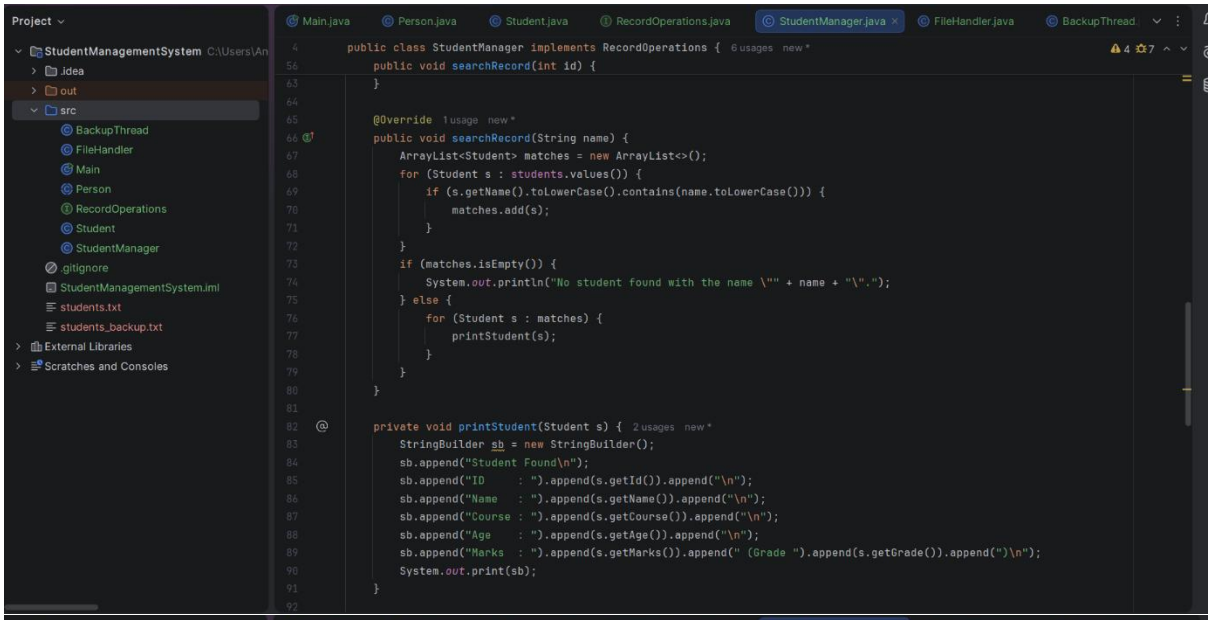


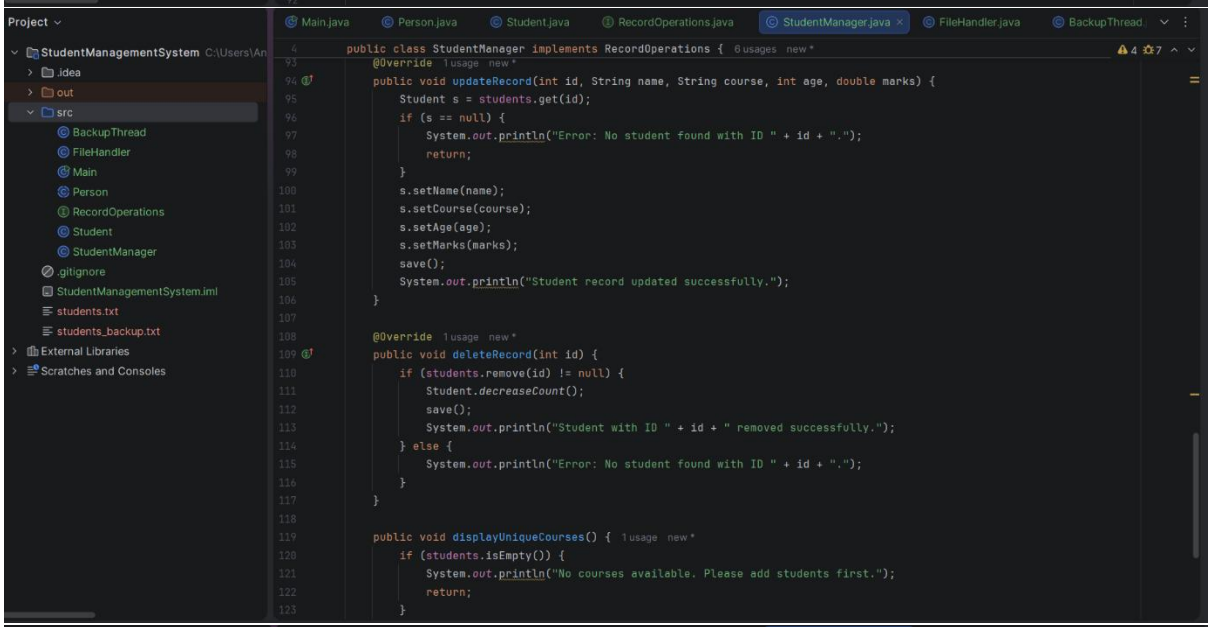
```
public class StudentManager implements RecordOperations {
    public void addRecord(Student student) {
    }

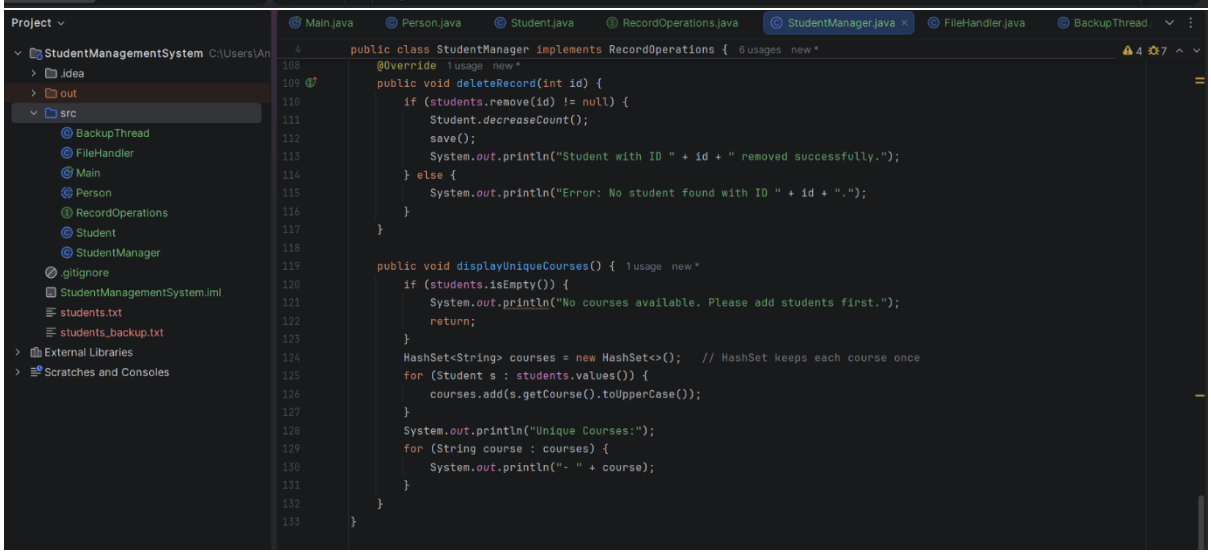
    @Override
    public void viewAllRecords() {
        if (students.isEmpty()) {
            System.out.println("No student records found.");
            return;
        }
        ArrayList<Student> list = getAllStudents();
        list.sort((Student a, Student b) -> Integer.compare(a.getId(), b.getId()));

        StringBuilder sb = new StringBuilder();
        sb.append(String.format("%-6s %-15s %-10s %-5s %-8s %s%n",
                "ID", "Name", "Course", "Age", "Marks", "Grade"));
        sb.append("-----------------------------------------------------------------\n");
        for (Student s : list) {
            sb.append(s).append("\n");
        }
        System.out.print(sb);
        System.out.println("Total students: " + Student.getTotalStudents());
    }

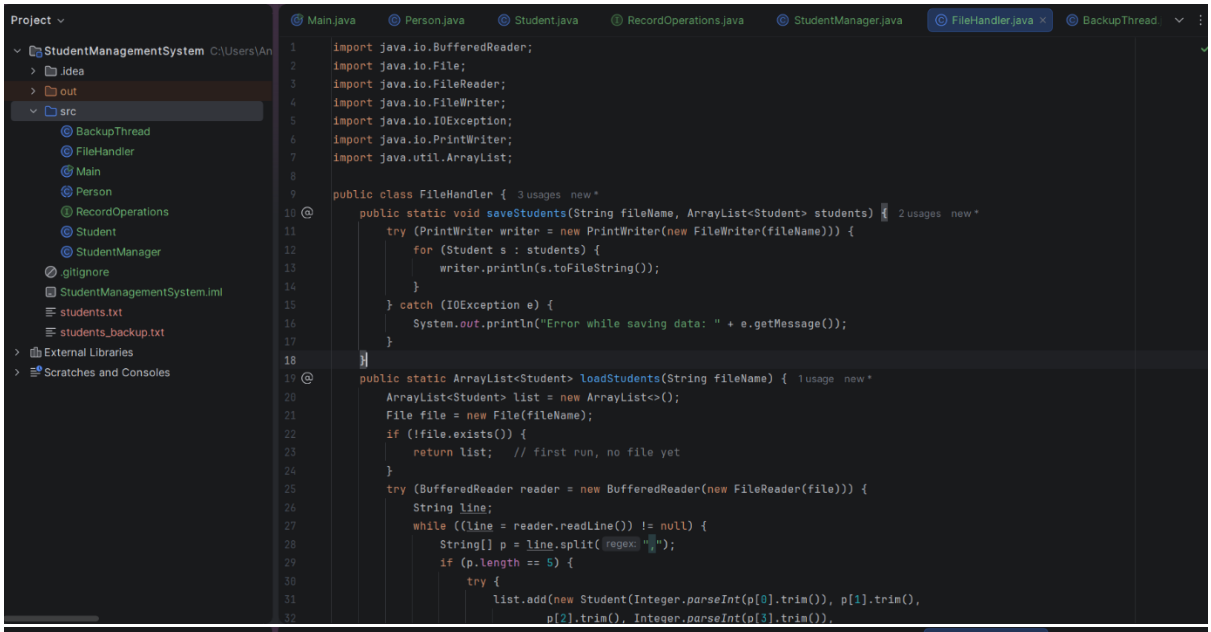
    @Override
    public void searchRecord(int id) {
        Student s = students.get(id);
        if (s == null) {
            System.out.println("Student with ID " + id + " not found.");
        } else {
            printStudent(s);
        }
```


Project
StudentManagementSystem C:\Users\An
.idea
out
src
BackupThread
FileHandler
Main
Person
RecordOperations
Student
StudentManager
.gitignore
StudentManagementSystem.iml
students.txt
students_backup.txt
External Libraries
Scratches and Consoles
Main.java
Person.java
Student.java
RecordOperations.java
StudentManager.java
FileHandler.java
BackupThread
public class StudentManager implements RecordOperations { 6 usages new *
public void searchRecord(int id) {
}
@Override 1 usage new *
public void searchRecord(String name) {
ArrayList<Student> matches = new ArrayList<>();
for (Student s : students.values()) {
if (s.getName().toLowerCase().contains(name.toLowerCase())) {
matches.add(s);
}
}
if (matches.isEmpty()) {
System.out.println("No student found with the name \"" + name + "\".");
} else {
for (Student s : matches) {
printStudent(s);
}
}
}
private void printStudent(Student s) { 2 usages new *
StringBuilder sb = new StringBuilder();
sb.append("Student Found\n");
sb.append("ID : ").append(s.getId()).append("\n");
sb.append("Name : ").append(s.getName()).append("\n");
sb.append("Course : ").append(s.getCourse()).append("\n");
sb.append("Age : ").append(s.getAge()).append("\n");
sb.append("Marks : ").append(s.getMarks()).append(" (Grade ").append(s.getGrade()).append(")\n");
System.out.print(sb);
}


Project
StudentManagementSystem C:\Users\An
.idea
out
src
BackupThread
FileHandler
Main
Person
RecordOperations
Student
StudentManager
.gitignore
StudentManagementSystem.iml
students.txt
students_backup.txt
External Libraries
Scratches and Consoles
Main.java
Person.java
Student.java
RecordOperations.java
StudentManager.java
FileHandler.java
BackupThread
public class StudentManager implements RecordOperations { 6 usages new *
@Override 1 usage new *
public void updateRecord(int id, String name, String course, int age, double marks) {
Student s = students.get(id);
if (s == null) {
System.out.println("Error: No student found with ID " + id + ".");
return;
}
s.setName(name);
s.setCourse(course);
s.setAge(age);
s.setMarks(marks);
save();
System.out.println("Student record updated successfully.");
}
@Override 1 usage new *
public void deleteRecord(int id) {
if (students.remove(id) != null) {
Student.decreaseCount();
save();
System.out.println("Student with ID " + id + " removed successfully.");
} else {
System.out.println("Error: No student found with ID " + id + ".");
}
}
public void displayUniqueCourses() { 1 usage new *
if (students.isEmpty()) {
System.out.println("No courses available. Please add students first.");
return;
}

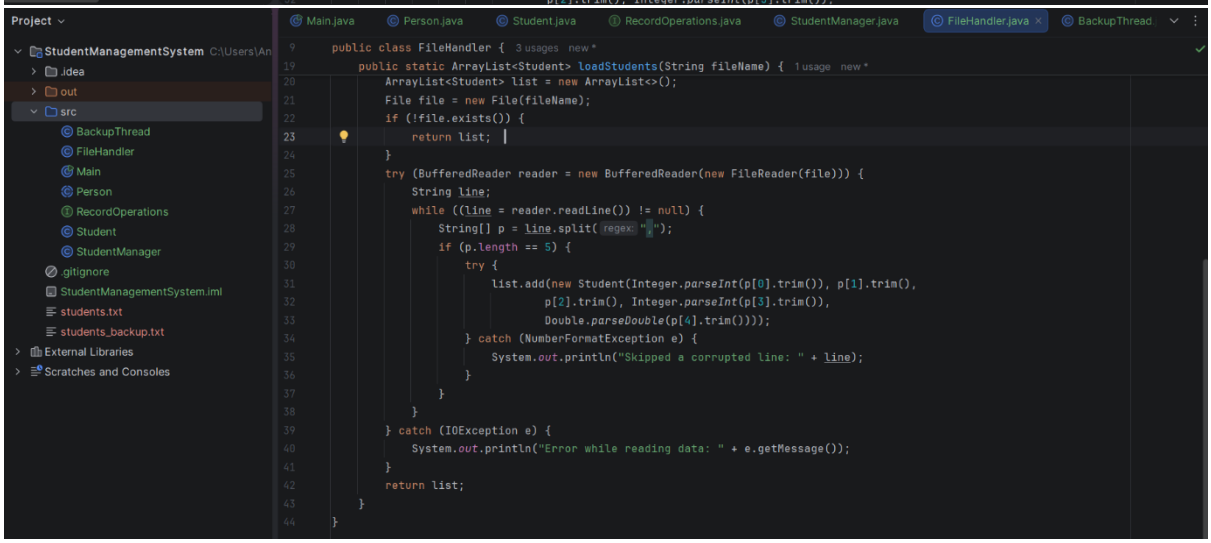

Project
StudentManagementSystem C:\Users\An
.idea
out
src
BackupThread
FileHandler
Main
Person
RecordOperations
Student
StudentManager
.gitignore
StudentManagementSystem.iml
students.txt
students_backup.txt
External Libraries
Scratches and Consoles
Main.java
Person.java
Student.java
RecordOperations.java
StudentManager.java
FileHandler.java
BackupThread
public class StudentManager implements RecordOperations { 6 usages new *
@Override 1 usage new *
public void deleteRecord(int id) {
if (students.remove(id) != null) {
Student.decreaseCount();
save();
System.out.println("Student with ID " + id + " removed successfully.");
} else {
System.out.println("Error: No student found with ID " + id + ".");
}
}
public void displayUniqueCourses() { 1 usage new *
if (students.isEmpty()) {
System.out.println("No courses available. Please add students first.");
return;
}
HashSet<String> courses = new HashSet<>(); // HashSet keeps each course once
for (Student s : students.values()) {
courses.add(s.getCourse().toUpperCase());
}
System.out.println("Unique Courses:");
for (String course : courses) {
System.out.println("- " + course);
}
}
}

FileHandler.java



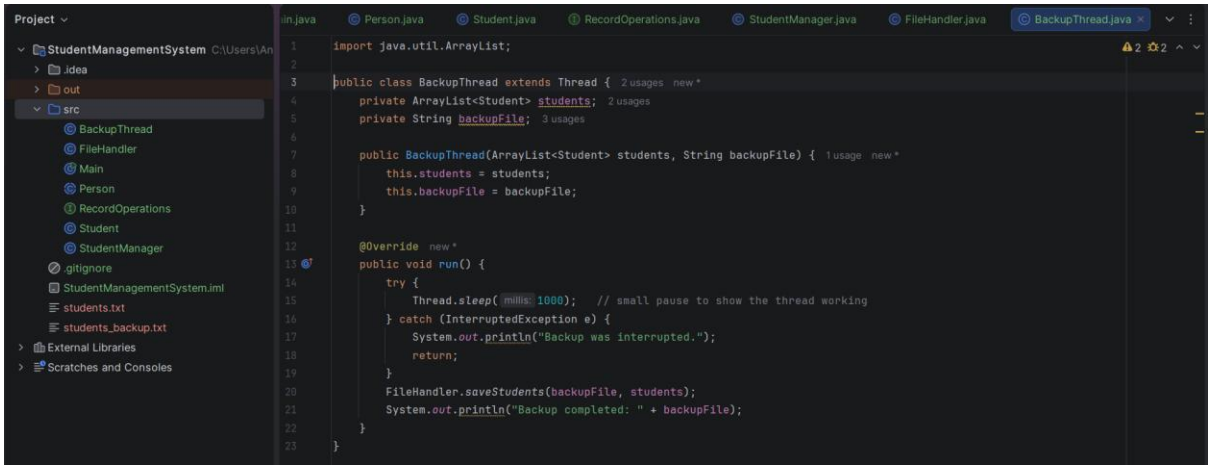
```java
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;

public class FileHandler {
    public static void saveStudents(String fileName, ArrayList<Student> students) {
        try (PrintWriter writer = new PrintWriter(new FileWriter(fileName))) {
            for (Student s : students) {
                writer.println(s.toFileString());
            }
        } catch (IOException e) {
            System.out.println("Error while saving data: " + e.getMessage());
        }
    }
    public static ArrayList<Student> loadStudents(String fileName) {
        ArrayList<Student> list = new ArrayList<>();
        File file = new File(fileName);
        if (!file.exists()) {
            return list;   // first run, no file yet
        }
        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] p = line.split(regex: ",");
                if (p.length == 5) {
                    try {
                        list.add(new Student(Integer.parseInt(p[0].trim()), p[1].trim(),
                                p[2].trim(), Integer.parseInt(p[3].trim()),
```



```java
public class FileHandler {
    public static ArrayList<Student> loadStudents(String fileName) {
        ArrayList<Student> list = new ArrayList<>();
        File file = new File(fileName);
        if (!file.exists()) {
            return list;
        }
        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] p = line.split(regex: ",");
                if (p.length == 5) {
                    try {
                        list.add(new Student(Integer.parseInt(p[0].trim()), p[1].trim(),
                                p[2].trim(), Integer.parseInt(p[3].trim()),
                                Double.parseDouble(p[4].trim())));
                    } catch (NumberFormatException e) {
                        System.out.println("Skipped a corrupted line: " + line);
                    }
                }
            }
        } catch (IOException e) {
            System.out.println("Error while reading data: " + e.getMessage());
        }
        return list;
    }
}
```

BackupThread.java



```java
import java.util.ArrayList;

public class BackupThread extends Thread {
    private ArrayList<Student> students;
    private String backupFile;

    public BackupThread(ArrayList<Student> students, String backupFile) {
        this.students = students;
        this.backupFile = backupFile;
    }

    @Override
    public void run() {
        try {
            Thread.sleep( millis: 1000);   // small pause to show the thread working
        } catch (InterruptedException e) {
            System.out.println("Backup was interrupted.");
            return;
        }
        FileHandler.saveStudents(backupFile, students);
        System.out.println("Backup completed: " + backupFile);
    }
}
```

OUTPUT:

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Program
===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 1
Enter Student ID: 101
Enter Name: Anagha
Enter Course: Java
Enter Age: 21
Enter Marks (0-100): 89
Student added successfully.

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 1
Enter Student ID: 102
Enter Name: Ishi
Enter Course: Python
Enter Age: 21
Enter Marks (0-100): 78
Student added successfully.

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 1
Enter Student ID: 103
Enter Name: Priya
Enter Course: Java
Enter Age: 21
Enter Marks (0-100): 95
Student added successfully.

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
```

```
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 1
Enter Student ID: 101
Error: Student ID 101 already exists.

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 2
ID     Name           Course             Age   Marks    Grade
-----------------------------------------------------------------
101    Anagha         Java               21    89.0     A
102    Ishi           Python             21    78.0     B
103    Priya          Java               21    95.0     A+
Total students: 3
```

```
===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 3
1. Search by ID
2. Search by Name
Enter option: 1
Enter Student ID: 102
Student Found
ID      : 102
Name    : Ishi
Course  : Python
Age     : 21
Marks   : 78.0 (Grade B)

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
```

```
Enter your choice: 3
1. Search by ID
2. Search by Name
Enter option: 2
Enter Student Name: Is
Student Found
ID     : 102
Name   : Ishi
Course : Python
Age    : 21
Marks  : 78.0 (Grade B)

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 4
Enter Student ID to update: 102
Enter new Name: Kavya
Enter new Course: Java
Enter new Age: 20
Enter new Marks (0-100): 85
Student record updated successfully.
```

```
===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 5
Enter Student ID to delete: 103
Student with ID 103 removed successfully.

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 6
Unique Courses:
- JAVA
```

```
===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 9
Error: Invalid menu choice. Please enter a number from 1 to 7.

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 7
Saving backup, please wait...
Backup completed: students_backup.txt
Thank you for using the Student Management System. Goodbye!


Process finished with exit code 0
```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Program
===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 1
Enter Student ID: 101
Enter Name: Anagha
Enter Course: Java
Enter Age: 21
Enter Marks (0-100): 89
Student added successfully.

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 1
Enter Student ID: 102
Enter Name: Ishi
Enter Course: Python
Enter Age: 21
Enter Marks (0-100): 78
Student added successfully.

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 1
Enter Student ID: 103
Enter Name: Priya
Enter Course: Java
Enter Age: 21
Enter Marks (0-100): 95
Student added successfully.

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
```

```
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 1
Enter Student ID: 101
Error: Student ID 101 already exists.

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 2
ID     Name            Course              Age   Marks    Grade
-----------------------------------------------------------------
101    Anagha          Java                21    89.0     A
102    Ishi            Python              21    78.0     B
103    Priya           Java                21    95.0     A+
Total students: 3
```

```
===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 3
1. Search by ID
2. Search by Name
Enter option: 1
Enter Student ID: 102
Student Found
ID     : 102
Name   : Ishi
Course : Python
Age    : 21
Marks  : 78.0 (Grade B)

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
```

```
Enter your choice: 3
1. Search by ID
2. Search by Name
Enter option: 2
Enter Student Name: Is
Student Found
ID     : 102
Name   : Ishi
Course : Python
Age    : 21
Marks  : 78.0 (Grade B)

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 4
Enter Student ID to update: 102
Enter new Name: Kavya
Enter new Course: Java
Enter new Age: 20
Enter new Marks (0-100): 85
Student record updated successfully.
```

```
 1. Add Student
 2. View All Students
 3. Search Student
 4. Update Student
 5. Delete Student
 6. Display Unique Courses
 7. Exit
 Enter your choice: 5
 Enter Student ID to delete: 103
 Student with ID 103 removed successfully.

 ===== Student Management System =====
 1. Add Student
 2. View All Students
 3. Search Student
 4. Update Student
 5. Delete Student
 6. Display Unique Courses
 7. Exit
 Enter your choice: 6
 Unique Courses:
 - JAVA
```

```
===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 9
Error: Invalid menu choice. Please enter a number from 1 to 7.

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Unique Courses
7. Exit
Enter your choice: 7
Saving backup, please wait...
Backup completed: students_backup.txt
Thank you for using the Student Management System. Goodbye!


Process finished with exit code 0
```

Challenges Faced:

- Keeping student IDs unique and handling wrong input without crashing the program were the main challenges.
- I solved them by checking the HashMap before adding, and by catching InputMismatchException and asking again.
- Another challenge was keeping the data after the program closed, which I solved by saving to a file and loading it on startup.

Conclusion:

- This project helped me join all the concepts from the internship in one application.
- I understood how classes, collections, exception handling, files and threads work together, and how to share a project using GitHub.